Здравствуйте, Юлия!

Код-ревьюер проверил ваш проект и оставил подробные комментарии. Ваша работа показывает, что вы освоили освоил ключевые концепции:

- У вас правильная структура проекта: созданы папки api, pages, tests, корректно настроены requirements.txt, pytest.ini и .gitignore.
- Вы понимаете паттерны: применили Page Object Model (POM), используете фикстуры PyTest в conftest.py для управления драйвером и данными.
- У вас сформировались навыки по работе CI/CD: файл .gitlab-ci.yml написан грамотно, с разделением на этапы, использованием кэша и сервисов.

При этом отметим, что автоматизированное тестирование в CI сейчас не работает. Тесты намеренно «заглушены», чтобы они не выполнялись в пайплайне, а просто сообщали об успехе.

Изучите подробную обратную связь от код-ревьюера ниже, внесите изменения в свой проект и отправьте его на проверку повторно.

Критические ошибки

❤️ В этой части ревью отражены грубые ошибки, из-за которых проект не выполняет свою главную задачу — полноценное тестирование в CI/CD.

1. Тесты не выполняются в CI

Самая главная проблема проекта заключается в том, что ни UI-, ни API-тесты фактически не работают в среде непрерывной интеграции (CI).

API-тесты
В файле `conftest.py` ваша фикстура `api_client` подменяет настоящий API-клиент на макет (`MockAPIClient`), если обнаруживает переменную окружения `CI`.

Фрагмент кода (`conftest.py`):

```
@pytest.fixture
def api_client():
    if os.getenv('CI'):
        # В CI возвращаем mock client
        class MockAPIClient:
```

```
            # ... (методы макета) ...
        return MockAPIClient()
    client = TandoorAPIClient()
    # ...
```

Это означает, что в GitLab-пайплайне вы тестируете не интеграцию с реальным сервисом Tandoor, а просто вызовы пустого объекта-заглушки. Цель интеграционного тестирования полностью теряется.

UI-тесты
Аналогично, в файле `tests/test_ui_meal_plan.py` каждый тест содержит проверку `if os.getenv('CI')`, после которой следует немедленный выход из теста с результатом `assert True`.

Фрагмент кода (`tests/test_ui_meal_plan.py`):

python

```
def test_create_meal_plan(self, driver, get_or_create_recipe):
    if os.getenv('CI'):
        assert True  # CI test passed
    else:
        # ... (логика теста) ...
```

Это приводит к тому, что в GitLab CI все UI-тесты помечаются как пройденные, хотя на самом деле ни одна проверка в браузере не была запущена.

Как исправить

Необходимо полностью удалить логику `if os.getenv('CI')` из фикстуры `api_client` и из всех ваших тестов. Тесты должны выполняться с использованием настоящих клиентов и драйверов как локально, так и в CI. Среда CI нужна именно для того, чтобы запускать реальные проверки в автоматическом режиме.

Узкие места

💛 В этой части ревью отражены узкие места в коде, которые снижают надёжность и эффективность тестов.

1. Зависимость тестов друг от друга

Ваши тесты не являются независимыми. Например, в `test_api_client.py` тест `test_delete_meal_plan` может выполниться успешно только если перед ним был запущен `test_create_meal_plan`, который создал план и сохранил его `id` в `pytest.plan_id`.

Фрагмент кода (`tests/test_api_client.py`):

```
def test_create_meal_plan(self, api_client,
get_or_create_recipe):
    # ...
    # Сохранить ID для удаления
    pytest.plan_id = plan['id']


def test_delete_meal_plan(self, api_client):
    if hasattr(pytest, 'plan_id'):
        result = api_client.delete_meal_plan(pytest.plan_id)
        # ...
```

Такой подход делает тесты хрупкими: изменение порядка запуска или падение одного теста приведёт к падению остальных.

Как исправить

Каждый тест должен сам создавать необходимые ему данные и удалять их после себя. Например, тест на удаление должен сначала создать объект, который он собирается удалить, и в конце проверить, что он действительно удалён.

---

2. Неэффективный логин в UI-тестах

В каждом UI-тесте вы заново выполняете вход в систему, вызывая `self.login(driver)`. Это сильно замедляет выполнение всего набора тестов.

Как исправить

Создайте фикстуру с областью видимости `module` или `session`, которая будет выполнять вход один раз для всех тестов в модуле или для всей сессии.

Что можно улучшить

💚 В этой части ревью даны правки, которые помогут сделать код ещё лучше.

1. Улучшение локаторов

Некоторые локаторы в ваших Page Object-ах основаны на тексте, например:

```
(By.XPATH, "//button[text()='Создать']")
```

Такие локаторы могут быть нестабильны, если текст на кнопке изменится. Старайтесь использовать более надёжные селекторы: уникальные `id`, `data-testid` или более конкретные CSS-селекторы.

---

2. Отсутствие кросс-проверки через API

В UI-тесте на удаление (`test_delete_meal_plan`) вы проверяете, что элемент исчез со страницы. Хорошей практикой было бы добавить после этого шаг, который через API-клиент проверяет, что этот объект действительно удалён из базы данных. Это сделает тест значительно надёжнее.

---

3. Неполное покрытие API-тестами

Ваши API-тесты не покрывают все методы клиента. Отсутствуют тесты для `import_recipe`, `delete_recipe` и, что важно, для сценариев с ошибками (например, попытка удалить несуществующий объект). Расширение покрытия сделает ваш API-клиент более надёжным.

Изучите рекомендации кодревьюера, внесите правки и сдайте вашу работу на проверку повторно.

Спасибо за задание, желаем вам успехов в дальнейшем обучении!